

1. Pydantic

- Pydantic is a Python library used for data validation and data parsing. It ensures that the data received by an application follows the defined types, structure, and rules.

2. Why do we need Pydantic?

- We need Pydantic to automatically validate and structure the data received by our API.
- It reduces the need to manually check data types and required fields using if-else conditions.

Keyword	Purpose	Example
BaseModel	Create a Pydantic model	class User(BaseModel)
Field	Add validation and constraints	Field(min_length=3)
EmailStr	Validate email format	email: EmailStr
HttpUrl	Validate URL	website: HttpUrl

3. What is BaseModel in Pydantic?

- BaseModel is a class provided by Pydantic that we inherit from to create data models with automatic data validation and parsing.

```
from pydantic import BaseModel

class UserStructure(BaseModel):
    name: str
    age: int
```

4. What is Field in Pydantic?

- Field() is a Pydantic function used to define validation rules and additional information for a field in a BaseModel.

Common Field() Parameters

Parameter	Meaning	Example
gt	Greater than	gt=18
ge	Greater than or equal to	ge=18
lt	Less than	lt=60
le	Less than or equal to	le=60
min_length	Minimum number of characters/items	min_length=3
max_length	Maximum number of characters/items	max_length=20
default	Default value	default=18
description	Description of the field	description="User age"
title	Title of the field	title="Age"
examples	Example values	examples=[25]

5. What is typing in Python?

- typing is a built-in Python module that provides tools for writing type hints.
- typing is a Python module that provides special types and tools to describe what kind of data a variable, function, or model should contain.

Commonly Used typing Keywords

Keyword	Meaning	Example
List	List containing a specific type	List[str]
Dict	Dictionary with specific key/value types	Dict[str, int]
Tuple	Tuple with specific types	Tuple[str, int]
Set	Set containing a specific type	Set[int]
Optional	Value can be a type or None	Optional[str]
Annotated	Add metadata/extra information to a type	Annotated[int, Field(gt=0)]